

Organização das cartas em colunas do jogo

No nosso jogo, as colunas estão organizadas em dois arrays que trabalham simultaneamente dentro de **struct jogo**.

1 - O espaço físico em pilhas [10] [52]

As colunas estão organizadas em forma de matriz

10 - Representa o número de colunas;

52 - Capacidade máxima de cada coluna no pior caso possível;

Se tivéssemos mais cartas do que espaço na coluna teríamos o erro de **segmentation fault**¹.

2 - O controlador lógico quantidade_pilhas[10]

Como a matriz tem 52 espaços para cartas por colunas, a parte lógica precisa distinguir o que são cartas do que são espaços vazios, e por isso usamos a o array **quantidade_pilhas**. Este array guarda o número inteiro de cartas ativas em cada coluna. Atua como o índice do topo da pilha

- Se **quantidade_pilhas[0] == 8 -->** significa que na primeira coluna temos cartas nas posições **pilhas[0][0]** até **pilhas[0][7]**.
- O índice 8 (**pilhas[0][8]**) será o primeiro espaço disponível se quisermos mover uma carta para lá.

3 - Como a lógica manipula esta organização

Organizar o jogo desta maneira torna os movimentos realizados por **moverCartas** bastante simples. Quando movermos uma carta a lógica faz o seguinte:

1. Vai à matriz da coluna A, copia as **struct carta** pretendidas, e cola-as na matriz da coluna B, começando no índice indicado por **quantidade_pilhasB**.
2. Atualiza os controladores **quantidade_pilhasB** e **quantidade_pilhasA**.

Quando retiramos uma carta da coluna, não apagamos cartas ou espaços da matriz. Apenas reduzimos a variável **quantidade_pilhas**, logo os espaços de memória lidos pelos computador diminuem ou aumentam de acordo com o movimento, e os restantes até 52 continuam como lixo de memória que é ignorado pelo computador.

4 - Motivo de escolha

- Escolhemos esta abordagem porque as regras do jogo têm limites físicos muito bem definidos (52 cartas) e assim garantimos que nunca irá haver falta de memória para todas as cartas do jogo.
- A matriz estática com o controlador lógico dá-me acesso instantâneo a qualquer carta, facilitando a cópia de sequências e evita os bugs complexos de gestão manual de memória por ponteiros.
- De facto há um ligeiro aumento na memória que é “desperdiçada” porém, devido à dimensão pequena do jogo, não é algo que afete em nada a performance e garante que o desenvolvimento do código é mais fácil e a possibilidade de erros seja menor.

¹ Acontece quando o programa tenta ler ou escrever num espaço de memória ao qual não tem acesso ou não existe